Agent Evaluation, Correctness, and Safety

Deep Dive into Kimi K3 — Chapter 31

Yin Yang

Foundation Books Project

The one idea of this chapter

A score measures a protocol, not a checkpoint alone;
a successful outcome does not certify the run that produced it.

The one idea of this chapter

A score measures a protocol, not a checkpoint alone;
a successful outcome does not certify the run that produced it.

Goal: qualify capability claims with matched comparisons and honest denominators, then check protocol, verifier, authority, effect, and shared-constraint obligations separately.

Roadmap

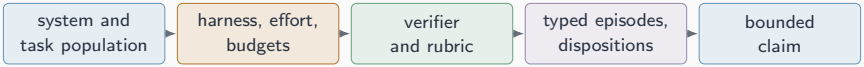
Measuring under a protocol

Correct execution

State, parallelism, and assurance

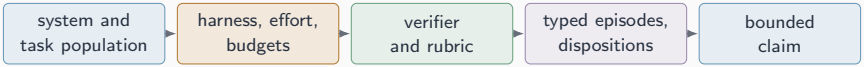
Measuring under a protocol

What a K3 result measures



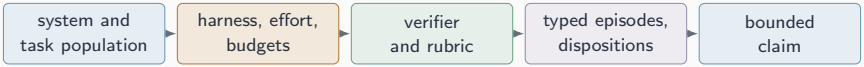
- K3 evaluates at max effort, temperature 1.0, with named harness assignments. [REPORTED: K3 Sec. 6.1.3]

What a K3 result measures



- K3 evaluates at max effort, temperature 1.0, with named harness assignments. [REPORTED: K3 Sec. 6.1.3]
- Paired repairs can tie in the mean while B improves and C regresses; a 1000-pair interval of about $[-7.6, +9.6]$ points still includes zero.

What a K3 result measures



- K3 evaluates at max effort, temperature 1.0, with named harness assignments. [REPORTED: K3 Sec. 6.1.3]
- Paired repairs can tie in the mean while B improves and C regresses; a 1000-pair interval of about $[-7.6, +9.6]$ points still includes zero.
- Retrieved, delivered, and used are three claims: Chapter 24's 5-to-6 of 24 gain is one seed on one pool. [OBSERVED: bounded search study]

Opportunity, denominators, and work

- Effort comparisons keep tokens, calls, turns, and time as separate coordinates; Pareto, not a bare sum.

[DERIVED: chapter teaching fixtures, reproduced by companion code]

Opportunity, denominators, and work

- Effort comparisons keep tokens, calls, turns, and time as separate coordinates; Pareto, not a bare sum.
- pass@k is opportunity in a finite pool ($n = 6$, $c = 2$: $1/3$, $3/5$); the submitted patch can still fail. Twelve dispatches give $7/9$ semantic and $7/12$ all-dispatch rates.

[DERIVED: chapter teaching fixtures, reproduced by companion code]

Opportunity, denominators, and work

- Effort comparisons keep tokens, calls, turns, and time as separate coordinates; Pareto, not a bare sum.
- pass@k is opportunity in a finite pool ($n = 6$, $c = 2$: $1/3$, $3/5$); the submitted patch can still fail. Twelve dispatches give $7/9$ semantic and $7/12$ all-dispatch rates.
- Swarm trace $(36, 18)$ versus serial $(32, 32)$: speedup $16/9$, work inflation $9/8$; wall time and a common-release join are separate checks.

[DERIVED: chapter teaching fixtures, reproduced by companion code]

Correct execution

From outcomes to execution obligations

- `studio-assure-001`: repair $(12, 8) + (100, 40) \mapsto (112, 48)$ in eight clean actions, at most twelve with two recoveries.

From outcomes to execution obligations

- `studio-assure-001`: repair $(12, 8) + (100, 40) \mapsto (112, 48)$ in eight clean actions, at most twelve with two recoveries.
- XTML validation and replay establish protocol form, not whether a pending allocation committed.

From outcomes to execution obligations

- `studio-assure-001`: repair $(12, 8) + (100, 40) \mapsto (112, 48)$ in eight clean actions, at most twelve with two recoveries.
- XTML validation and replay establish protocol form, not whether a pending allocation committed.
- Verifier isolation blocks tampering, not coverage gaps; exposure rules moved 7 of 12 sealed cases out of the clean set. [OBSERVED: controlled leakage study]

From outcomes to execution obligations

- `studio-assure-001`: repair $(12, 8) + (100, 40) \mapsto (112, 48)$ in eight clean actions, at most twelve with two recoveries.
- XTML validation and replay establish protocol form, not whether a pending allocation committed.
- Verifier isolation blocks tampering, not coverage gaps; exposure rules moved 7 of 12 sealed cases out of the clean set. [OBSERVED: controlled leakage study]
- Untrusted content cannot enlarge a grant; authority is checked at commitment, serialized against revocation.

State, parallelism, and assurance

Unknown effects, shared limits, and a scorecard

- “Preview unfinished” hides two histories; a bounded receipt query either resolves the effect or stops with the request pending.

Unknown effects, shared limits, and a scorecard

- "Preview unfinished" hides two histories; a bounded receipt query either resolves the effect or stops with the request pending.
- Two workers reading zero allocate two jobs; a service-held mission reservation excludes that interleaving.

Unknown effects, shared limits, and a scorecard

- "Preview unfinished" hides two histories; a bounded receipt query either resolves the effect or stops with the request pending.
- Two workers reading zero allocate two jobs; a service-held mission reservation excludes that interleaving.
- A safeguard scorecard keeps four denominators: useful completion, harmful effects, unresolved outcomes, unnecessary blocks.

Unknown effects, shared limits, and a scorecard

- "Preview unfinished" hides two histories; a bounded receipt query either resolves the effect or stops with the request pending.
- Two workers reading zero allocate two jobs; a service-held mission reservation excludes that interleaving.
- A safeguard scorecard keeps four denominators: useful completion, harmful effects, unresolved outcomes, unnecessary blocks.

This closes Volume II. Next: Chapter 32's From Basic CUDA to Kimi Kernels, opening Volume III, Kernels and System Co-Design.